ADVENTIST UNIVERSITY OF CENTRAL AFRICA

Name: Uwayezu Bienvenu

Student ID: 101362

GitHub: https://github.com/bienvenu123/analyticd_exe

AUCA | ANALYTICS LAB

Distributed Multi-Model Analytics for E-commerce Data

Technical project report with evidence screenshots

MongoDB | Apache HBase | Apache Spark

Summary of results

This report documents a multi-model analytics pipeline for synthetic e-commerce data. **MongoDB** stores bounded business entities, **HBase** stores time-ordered page events, and **Spark** performs batch, cross-store and simulated-stream analysis. Each claim is backed by a screenshot or chart in this report.

Scope	Implemented evidence
MongoDB	Sample loaded locally; product-popularity and category-revenue pipelines executed.
HBase	Local HBase 2.6.6 and Thrift started; 2 tables created; 99,873 page-view rows and 24,881 product/day rows loaded.
Spark	All 2,000,000 sessions and 500,000 transactions summarised; MongoDB and HBase joined on the sample.
Visuals	Four charts, a simulated stream, and an interactive dashboard over MongoDB and HBase.

Scale distinction. The full generator produced 10,000 users, 5,000 products, 25 categories, 2,000,000 sessions and 500,000 transactions, and Spark batch ran over those files. MongoDB, HBase and the cross-store join use a manageable assignment sample of 1,000 users, 500 products, 10,000 sessions and 2,500 transactions.

Measure	Full-scale Spark batch	Assignment sample
Users / products	10,000 / 5,000	1,000 / 500
Sessions	2,000,000	10,000
Transactions	500,000 generated; 439,067 fulfilled	2,500 generated
Converted sessions	405,887 (20.3%)	2,057 (20.6%)
Revenue	399,384,245.22 fulfilled	Leading category cat_009: 274,758.91
HBase event rows	not loaded (sample only)	99,873 page views; 24,881 product/day rows

1. Architecture and data

Each workload goes to the model that fits its access pattern. MongoDB holds catalog, profile, session and transaction documents. HBase stores high-volume page-view events for keyed time-range retrieval. Spark reads the generated JSON files for batch analytics. In integration mode, profiles and fulfilled transactions come from MongoDB (PyMongo) and event rows from HBase (HappyBase); these are staged as JSON, then joined and aggregated with Spark DataFrames.

Stage	Technology	Purpose
Source	JSON files	Synthetic users, products, categories, sessions, transactions.
Operational documents	MongoDB	Nested records, selective indexes, lookup and group pipelines.
Event retrieval	HBase	Sparse page events keyed for a user-first recent-activity scan.
Analytics	Spark	Distributed joins, cohort aggregation, co-purchase, integrated metrics.
Presentation	Matplotlib + PDF	Four static plots and findings for a reviewable report.

Data preparation

The supplied `dataset_generator.py` is an exported notebook JSON object, not an executable module. `run_assignment_generator.py` extracts its code cell, overrides the scale values and runs the code unchanged. The same runner produced both the 10,000-session sample and the full 2,000,000-session dataset, written as bounded chunks (25,000 sessions and 10,000 transactions per file, i.e. 80 and 50 files).

Pipeline: generate JSON, load MongoDB and HBase (optional), run Spark normalisation and analysis, write Parquet/CSV aggregates and PNG charts, build the report.

2. MongoDB and HBase design

MongoDB collections

Collection	Document boundary and indexes
products	One document per product; bounded price history. Unique product_id; category_id + is_active.
users	One document per user with nested geo and registration fields. Unique user_id.
transactions	One document per order with embedded items. Indexes: transaction_id; timestamp + status; user_id + timestamp.
sessions	One document per session with nested page_views, device, cart. Index user_id + start_time descending.

The **product-popularity** pipeline filters fulfilled orders, unwinds items, groups units and revenue, looks up product names and ranks the top 20. The **category-revenue** pipeline unwinds items, joins products and groups revenue, units and distinct orders by category. The loaded database `auca_ecommerce` contains categories, products, sessions, transactions and users (Figure 4, page 5).

HBase tables

Table	Row key	Column families
user_events	user_id | reverse_epoch_ms | session_id | event_index	s: session attributes; e: page type, product, category, duration, timestamp
product_daily	product_id | yyyyMMdd	m: product views, cart-add units, purchased units, revenue in cents

The user_events key clusters each user's rows and sorts newest first because the epoch is reversed, which supports a contiguous range scan for one user. product_daily supports product and date-range queries. Prefixing user IDs in a globally ordered table can hotspot a few users; salting or region partitioning spreads write load at the cost of fan-out reads.

HBase execution

Standalone HBase 2.6.6 and the Thrift gateway ran locally on ports 16010 and 9090. The loader processed 10,000 sessions for 1,000 users and wrote **99,873** page-view rows to user_events; a table scan returned the same count. It also loaded **24,881** product/day rows. A user-prefix query for user_000042 returned 20 event rows, confirming the reverse-time scan.

Why both stores

MongoDB suits flexible, bounded entities and multi-field lookups. HBase suits sparse, append-heavy events and predictable row-key scans. Spark handles broad joins and reductions rather than point retrieval. Duplicating every record in both stores would add cost without improving the stated access patterns.

The HBase Master page (Figure 6, page 5) independently confirms the design: user_events carries families e and s, product_daily carries m, on a single-node 2.6.6 standalone with 4 regions in total (2 user tables plus hbase:meta and hbase:namespace). The HBase root directory is a temporary folder, so data does not persist after clean-up; this is fine for a demonstration but not for production.

Evidence: full-scale generation and Spark results

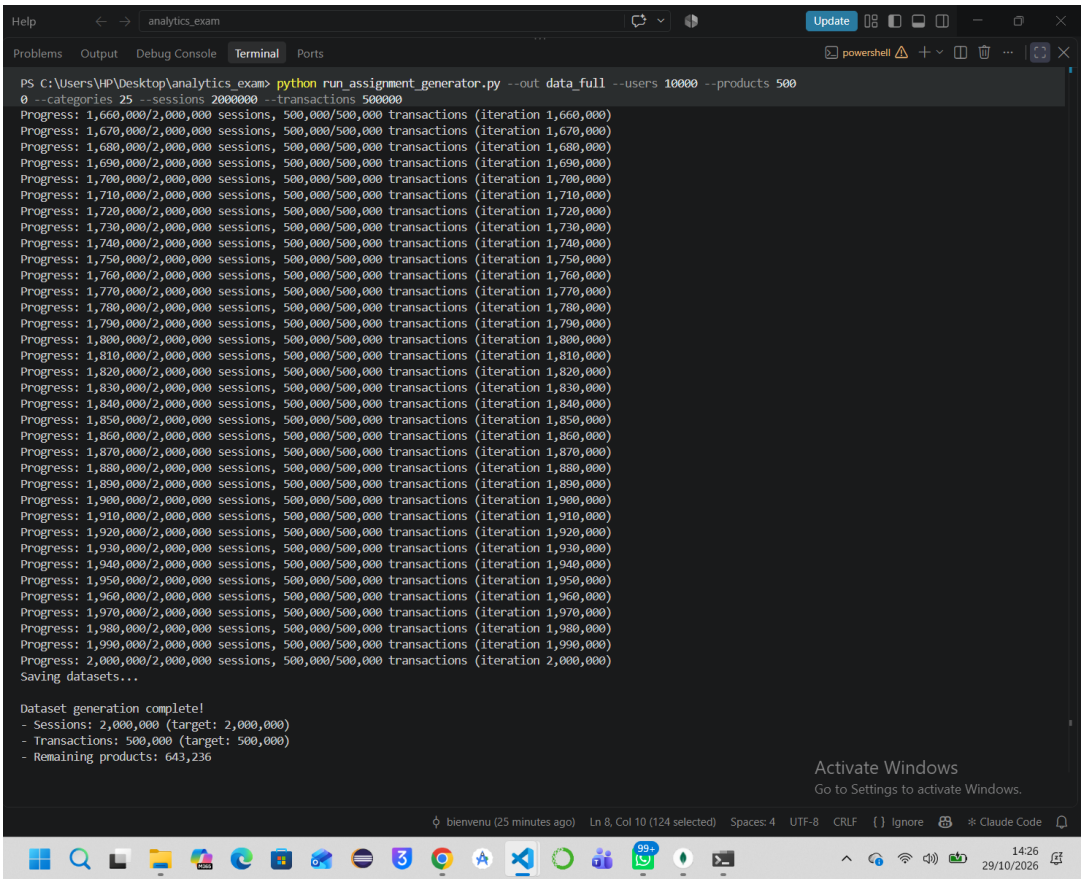


Figure 1. Full-scale generator run (29/10/2026, 14:26). Command: run_assignment_generator.py --out data_full --users 10000 --products 5000 --categories 25 --sessions 2000000 --transactions 500000. Both targets reached exactly (2,000,000 sessions; 500,000 transactions).

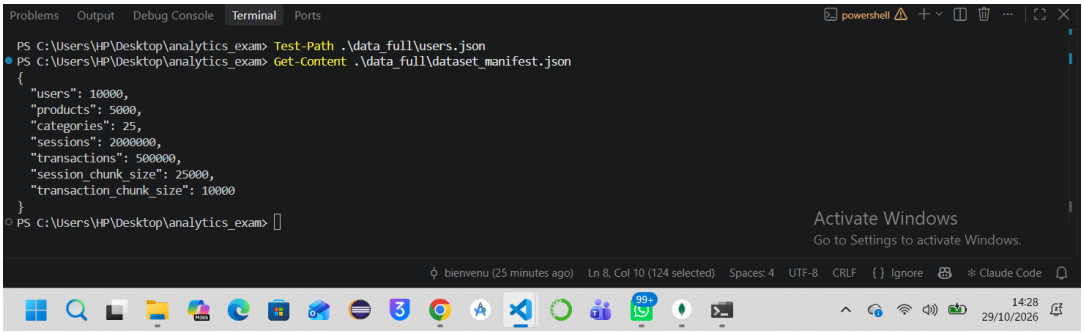


Figure 2. dataset_manifest.json (14:28): 10,000 users, 5,000 products, 25 categories, 2,000,000 sessions, 500,000 transactions; chunk sizes 25,000 and 10,000, which fixes 80 session files and 50 transaction files.

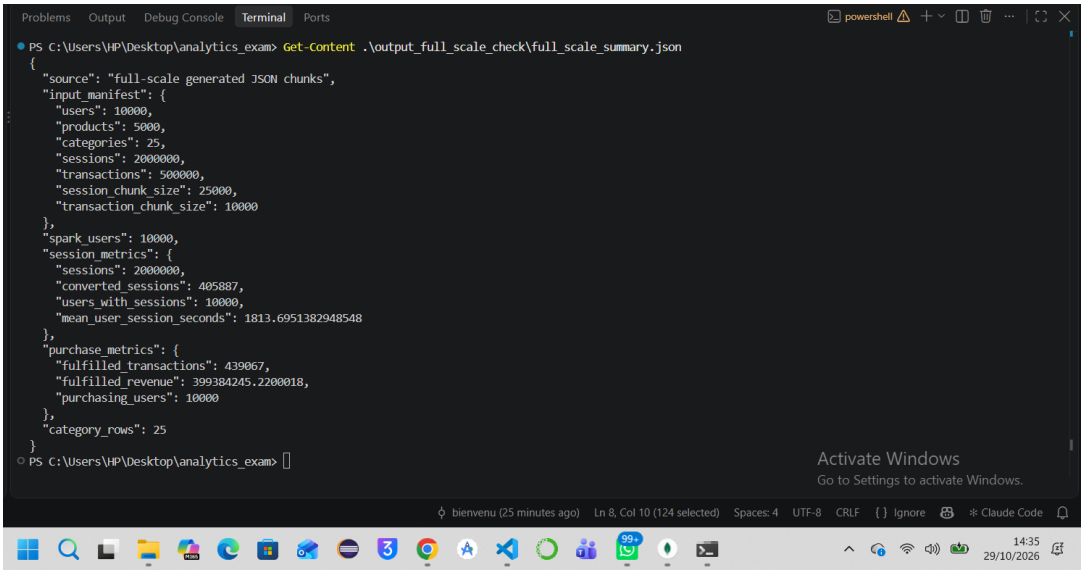


Figure 3. full_scale_summary.json (14:35): 405,887 converted sessions (20.3%); 439,067 fulfilled transactions (87.8%); fulfilled revenue 399,384,245.22 (about 909.62 per transaction); mean user session 1,813.7 s (about 30.2 min); 10,000 users all with sessions and purchases.

Evidence: MongoDB and HBase

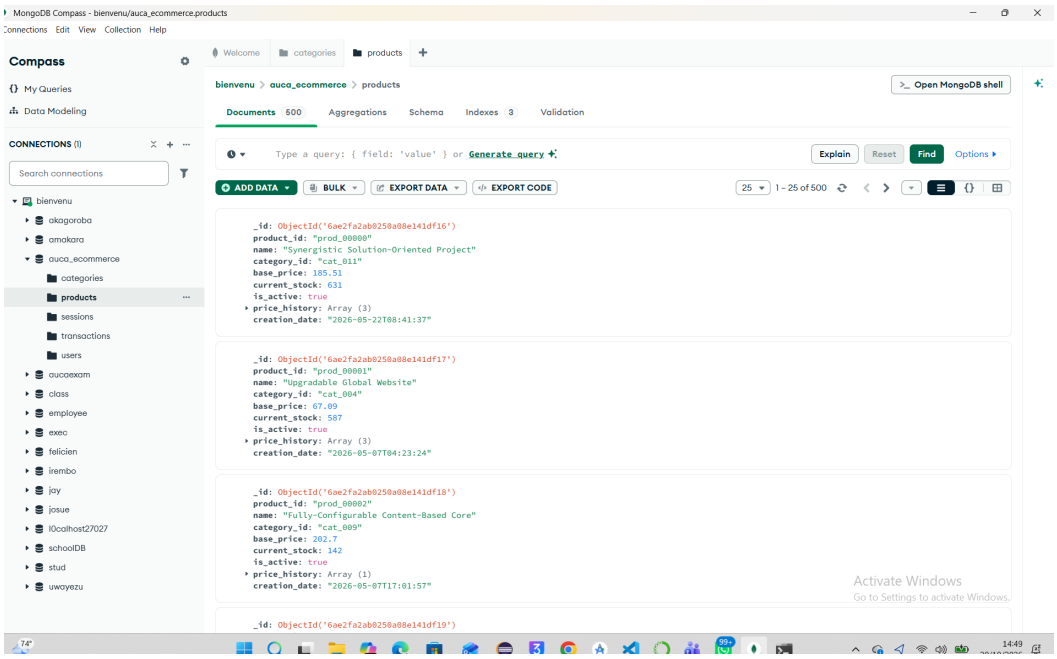


Figure 4. MongoDB Compass, auca_ecommerce.products (14:49): 500 documents, 3 indexes; fields product_id, name, category_id, base_price, current_stock, is_active, price_history, creation_date. The categories collection holds the 15 sample categories (full manifest: 25).

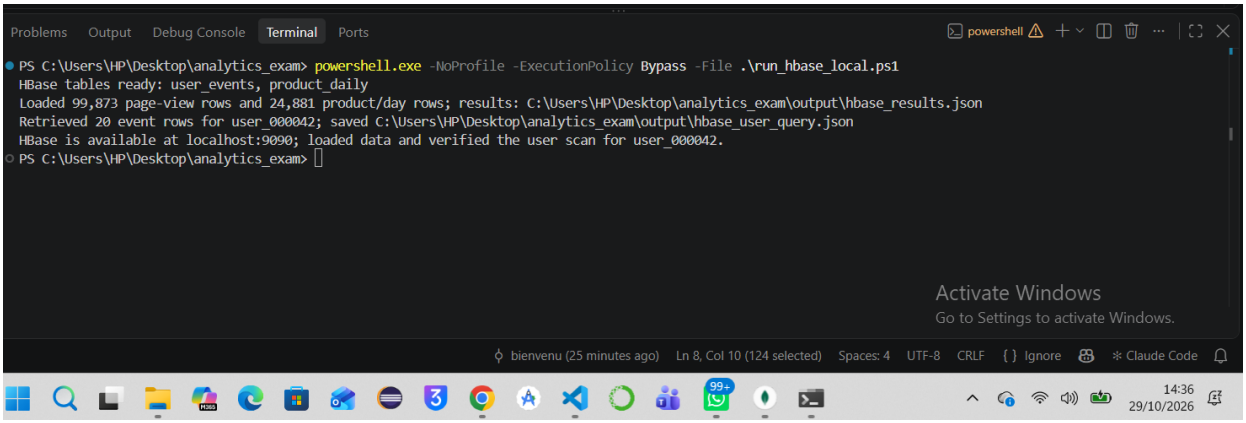


Figure 5. run_hbase_local.ps1 output (14:36): tables user_events and product_daily ready; 99,873 page-view rows and 24,881 product/day rows loaded; 20 event rows retrieved for user_000042.

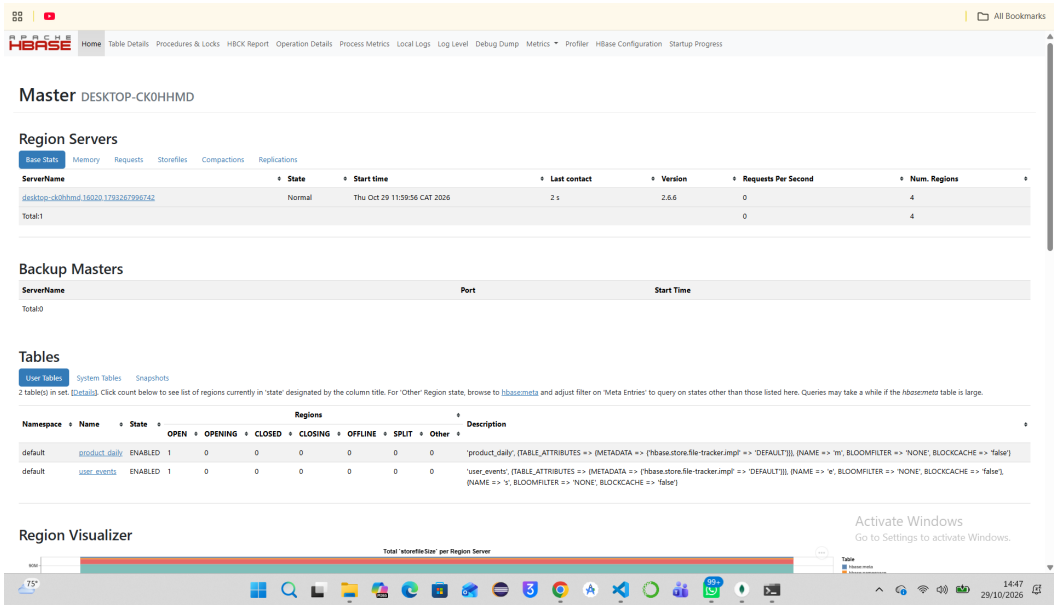


Figure 6. HBase Master UI (14:47): version 2.6.6, 1 region server (state Normal), 4 regions, both user tables ENABLED; product_daily family m, user_events families e and s.

3. Spark processing and integration

The PySpark application reads multi-line JSON arrays, deduplicates primary IDs, parses event timestamps, explodes fulfilled transaction line items, and uses DataFrame joins and SQL. The co-purchase job self-joins distinct product IDs per order and counts co-occurrence (minimum support of two). Cohort spend groups users by registration month and purchase month. Spark SQL joins line items to categories for revenue, units and order counts.

Integrated business question

Which customer groups combine higher spending with stronger engagement, and where do sessions leave the purchase funnel? Spark joins user profiles, session counts and durations, and fulfilled order totals, then bands users into frequent (5+ orders), repeat (2-4), one-time (1) and no-purchase groups. The verified run joined 1,000 MongoDB profiles with 10,000 HBase sessions. CLV is observed cumulative revenue, not a forecast.

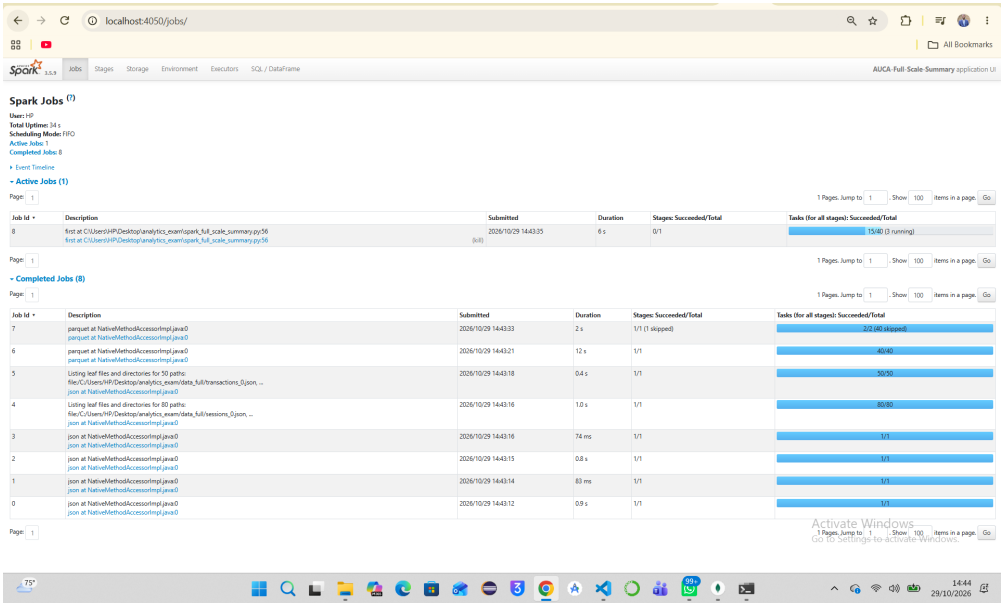


Figure 7. Spark UI, application AUCA-Full-Scale-Summary (14:44, Spark 3.5.9, local mode): Job 4 lists 80 session paths and Job 5 lists 50 transaction paths, matching the manifest. Job 8 was still running at capture; final numbers come from Figure 3.

Assignment-derived findings

Measure	Observed	Interpretation
Sessions / transactions	10,000 sessions; 2,500 transactions	Reduced-scale generated data.
Funnel	9,885 product-view; 6,732 cart; 2,057 converted	20.6% of sessions converted; 30.6% of cart sessions.
Purchase frequency	77 frequent, 560 repeat, 248 one-time, 115 no purchase	Bands use fulfilled transaction history.
Category revenue	cat_009 leads at 274,758.91; cat_012 next at 191,715.93	MongoDB and Spark independently agree.

MongoDB ranked prod_00292 first by units (40 units, 19 orders, 12,685.20 item revenue). Fulfilled orders gave 3,472 distinct co-purchase pairs before the support filter; the most frequent pair appeared in three orders. These are synthetic, reduced-scale results, not validated recommendations.

4. Visual findings

Charts are generated by spark_analytics.py at the assignment scale.

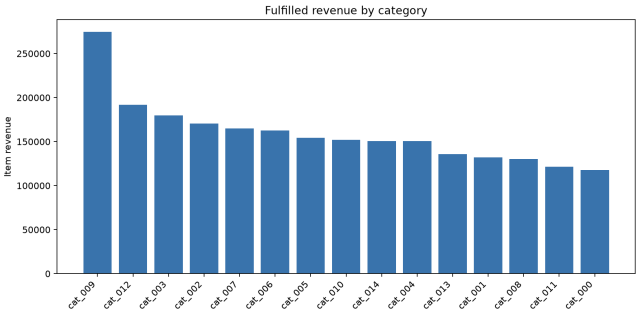


Figure 8. Fulfilled revenue by category. cat_009 earns about 1.4 times cat_012; the remaining 13 categories decline gradually (cat_000 about 117,000).

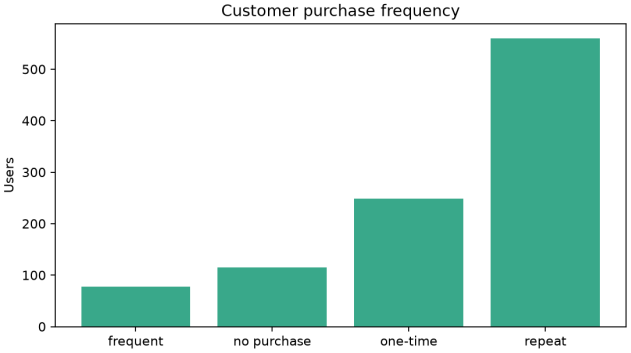


Figure 9. Purchase frequency: 77 frequent (7.7%), 560 repeat (56.0%), 248 one-time (24.8%), 115 no purchase (11.5%).

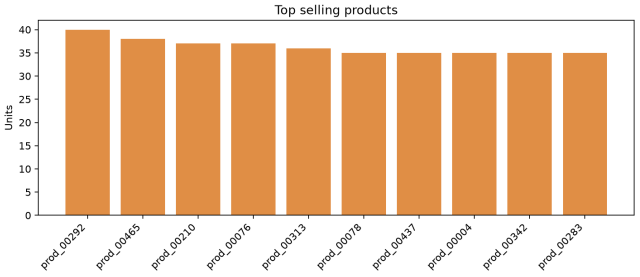


Figure 10. Top products by units: a narrow 35 to 40 unit band, led by prod_00292 (40), prod_00465 (38). Ranking by revenue differs: prod_00078 leads at 18,068.40.

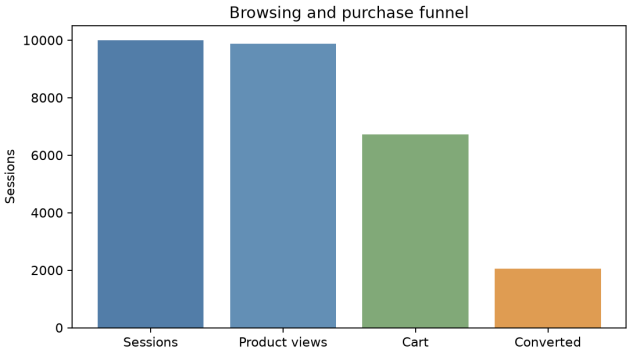


Figure 11. Funnel: 10,000 sessions, 9,885 product views, 6,732 carts, 2,057 conversions. Largest drop is product view to cart (31.9% lost).

Reading the charts

Revenue is concentrated in one category, and more than half of sampled users are repeat buyers. Browsing is not the bottleneck: nearly every session reaches a product page, and the sharpest loss is between product view and cart. At full scale conversion is 20.3%, close to the sample's 20.6%, suggesting the sample is representative on this measure.

Simulated streaming

A Spark Structured Streaming job uses the rate source to generate synthetic page events, groups them into 5-second event-time windows by page type and category, and writes each microbatch. The verified 40-second run (10 rows per second, about 400 events) produced **10 microbatches** and **346 windowed groups**. The 346 are distinct (window, page type, category) combinations, not events. This is a simulation, not a live source.

5. Interactive dashboard

The "Commerce intelligence" dashboard has five tabs (Overview, Sales explorer, Customer 360, HBase activity, Data explorer) with live connections to MongoDB (localhost:27017) and the HBase Thrift gateway (localhost:9090), 10-second auto-refresh, and filters for date (31 Jul to 29 Oct 2026), status and category.

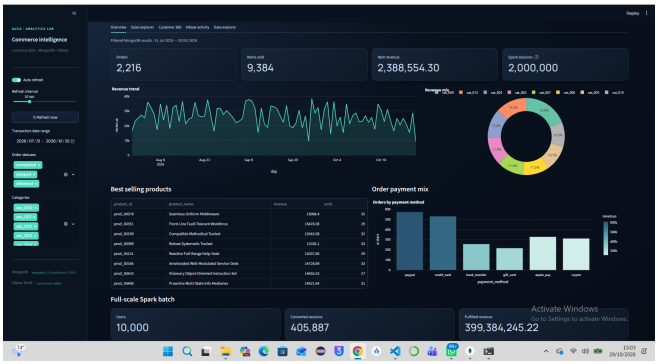


Figure 12. Overview (15:03): 2,216 orders, 9,384 items, item revenue 2,388,554.30; the Spark strip shows 10,000 users, 405,887 converted sessions, 399,384,245.22 revenue. cat_009 leads at 18.9% of revenue.

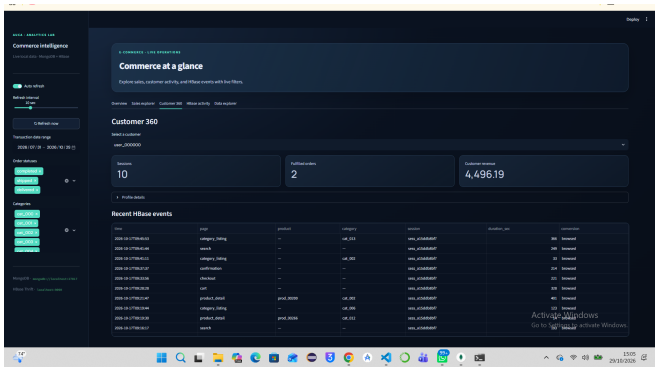


Figure 13. Customer 360 for user_000000 (15:05): 10 sessions, 2 fulfilled orders, revenue 4,496.19 from MongoDB, with recent events from an HBase row-key scan (newest first).

6. Scalability, limitations and reproduction

Scalability. At full scale use bulk writes and retry-safe IDs for MongoDB, and shard only after choosing keys aligned to common filters. Choose HBase event keys that distribute writes while keeping user-level scan locality. In Spark, partition by date or stable key, avoid collecting unbounded data to the driver, broadcast only small dimensions, and write Parquet to avoid repeated JSON parsing.

Limitations. This is a local, single-machine run, not a cluster benchmark. MongoDB, HBase and the direct cross-store join use the sample, while Spark batch used the full files. The generator has no age field, so purchase-frequency and registration cohorts replace age segmentation. Session-linked and standalone transactions must be distinguished for funnel attribution. Inventory is a snapshot, not a stock ledger. The MongoDB pipelines, co-purchase count, cross-store join and streaming run are stated from analysis output without screenshots. All data are synthetic, so findings demonstrate the workflow and are not market estimates.

Reproduction. Install requirements.txt, start local HBase with the project script, then:

```
python run_assignment_generator.py --out data_full --users 10000 --products 5000 --categories 25 --sessions 2000000
--transactions 500000
spark-submit --master local[2] spark_full_scale_summary.py --data data_full --schema-sample data_assignment --out
output_full_scale_summary
spark-submit --master local[2] spark_analytics.py --source-mode stores --out output
spark-submit --master local[2] spark_streaming_simulation.py --duration-seconds 40 --rows-per-second 10
```

Conclusion. MongoDB fits bounded business entities, HBase fits time-ordered sparse events and product/day metrics, and Spark covers batch, cross-store and simulated stream processing. Full-scale batch ran on the generator defaults; the verified database loads and direct join use the assignment sample.